Object-Oriented Principles in Two Languages: A Ride Sharing System in C++ and Smalltalk

Saurabh Dahale

Department of Information Technology, University of the Cumberlands

MSCS-632-M30, Advanced Programming Languages

November 7, 2026

Object-Oriented Principles in Two Languages: A Ride Sharing System in C++ and Smalltalk

The same ride sharing system was implemented twice, once in C++ and once in GNU Smalltalk, under a specification fixing the classes, fare rules and exact output text written before either program. A script builds and runs both and compares the results, identical line for line apart from one section C++ cannot produce, so every remaining difference is the language. Both define a Ride base class, two subclasses that price differently, a Driver owning a private ride list, and a Rider recording requests. Source: https://github.com/saurabh-dahale/Ride-Sharing-System.

Encapsulation

C++ makes encapsulation a decision. Driver declares assignedRides private, so the collection changes only through addRide and is read only through getDriverInfo. Removing that one keyword would expose the field and nothing would object. The protection is checked before the program exists. Uncommenting the direct access in main produces a compile error naming the member and where it was declared private.

Smalltalk removes the decision. Instance variables are private to the object by construction and no public option exists (Goldberg & Robson, 1983), so Driver's state is unreachable from outside whether or not the author thought about it. Enforcement arrives later. Sending assignedRides to a driver with no accessor is accepted when the file loads and fails only on execution, raising MessageNotUnderstood. Sebesta (2019) frames this as when errors are detected rather than whether a language can express the constraint. Smalltalk guarantees more by default and reports violations later, C++ guarantees only what is declared and reports violations first. That check is the section C++ cannot print, because the equivalent line never compiles.

Inheritance

C++ derives StandardRide and PremiumRide publicly from Ride, whose data members are protected rather than private so a subclass fare() reads distance directly while outside code cannot. Declaring fare() pure virtual makes Ride abstract and instantiating it is a compile error. Smalltalk expresses the same hierarchy with Ride subclass: StandardRide, but abstractness is a convention, and Ride implements fare as self subclassResponsibility, which raises only when a missing override is reached. The languages also differ in what a class is. The fare constants in Smalltalk are class-side methods, because a class is itself an object that responds to messages, and subclasses inherit them. C++ has no equivalent, so the same constants are static members.

Polymorphism

Both programs store four rides in one collection and call fare and rideDetails without testing the type. In C++ the collection is a vector of shared pointers to Ride and each call reaches the override for the object's actual class through the virtual table. The binding happens at run time, but what the vector may hold is fixed at compile time. Smalltalk places no such bound. An OrderedCollection accepts any object and a message is dispatched by looking up the selector in the receiver's class at send time, so the elements need not share a superclass. Any object answering fare and rideDetails would work in the same loop.

One structure is identical in both. rideDetails is defined once in Ride and never overridden, yet behaves differently per subclass because it calls the two overridden methods. That is the Template Method pattern (Gamma et al., 1994), and it works the same way under virtual dispatch and message sending.

References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

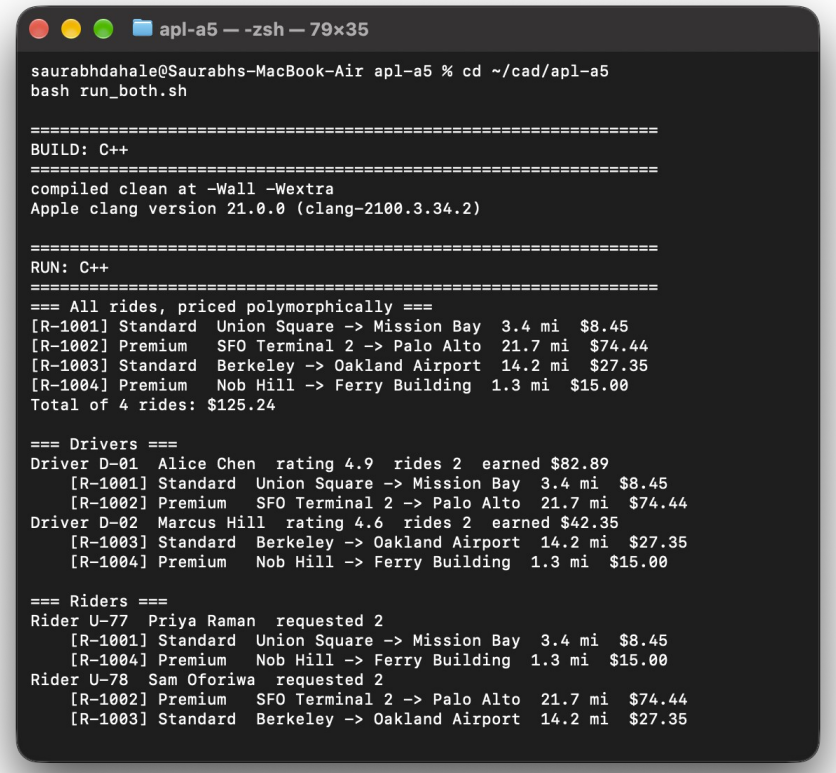
Goldberg, A., & Robson, D. (1983). *Smalltalk-80: The language and its implementation*. Addison-Wesley.

Sebesta, R. W. (2019). *Concepts of programming languages* (12th ed.). Pearson.

Appendix: Sample Output and Source

Figure 1

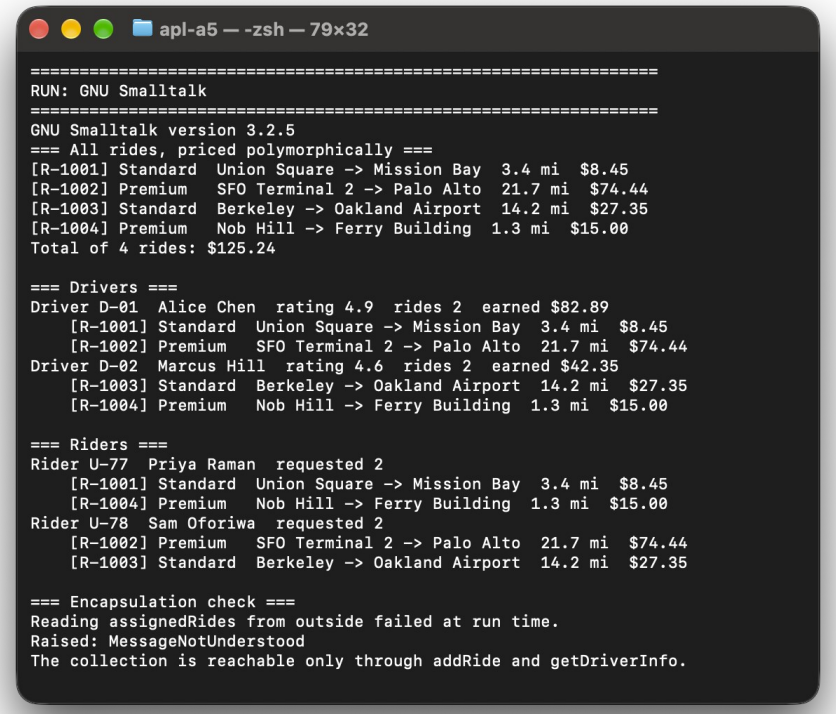
C++ Build and Run



Note. The program compiles clean at -Wall -Wextra under Apple clang. The four rides are priced through the same loop, which never asks what kind of ride it holds.

Figure 2

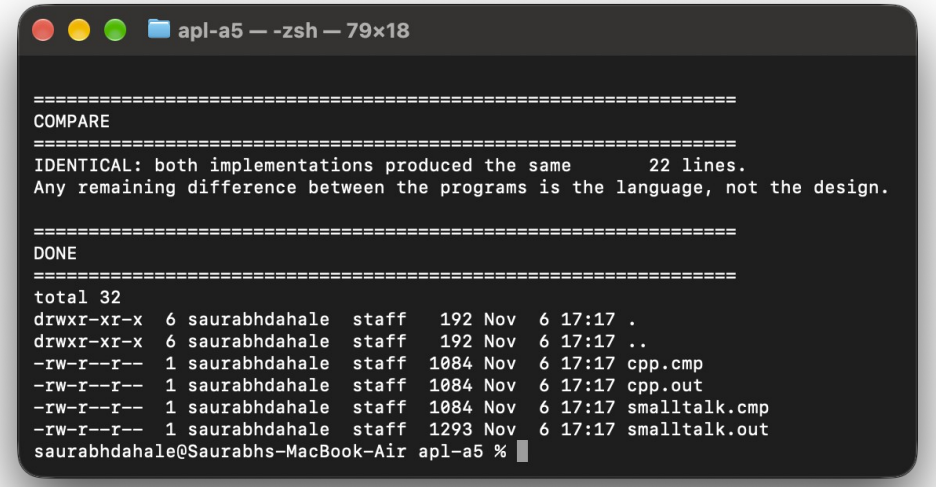
GNU Smalltalk Run



Note. The same three sections, identical to the C++ output. The fourth section has no C++ counterpart. Reading a driver's ride collection from outside raises MessageNotUnderstood at run time, where the equivalent C++ line is rejected by the compiler.

Figure 3

Comparison of the Two Outputs



Note. The script strips the Smalltalk-only section and diffs the rest. All 22 lines match.

Figure 4

Smalltalk Ride Class

ridesharing.st

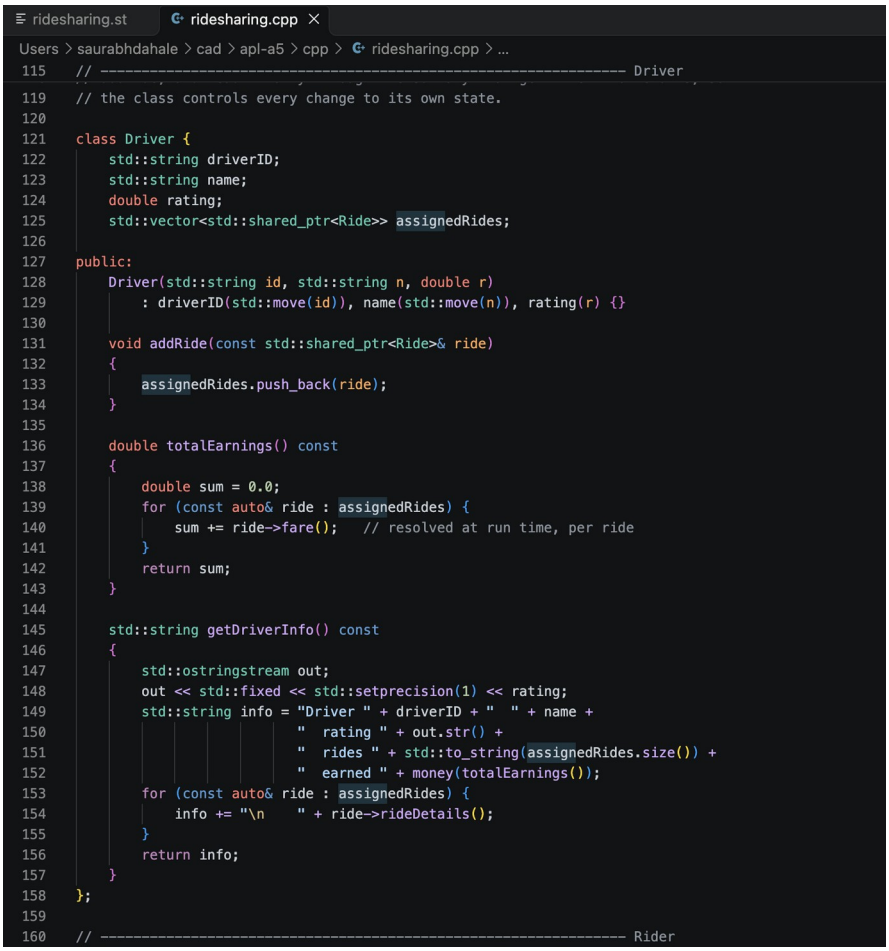
Users > saurabhdahale > cad > apl-a5 > smalltalk > ridesharing.st

```
49
50  Inheritance. Ride is the base class holding the state every ride has.
51
52  Encapsulation. In Smalltalk instance variables are private to the object by
53  construction. There is no public option to choose, so the four variables below
54  are reachable only from methods of Ride and its subclasses.
55
56  Polymorphism. fare is declared here as subclassResponsibility, so every
57  concrete ride must supply its own pricing rule, and a caller can price a ride
58  without knowing which kind it holds."
59
60  Object subclass: Ride [
61      | rideID pickupLocation dropoffLocation distance |
62
63      Ride class >> id: anId from: aPickup to: aDropoff distance: aDistance [
64          ^self new setId: anId from: aPickup to: aDropoff distance: aDistance
65      ]
66
67      setId: anId from: aPickup to: aDropoff distance: aDistance [
68          rideID := anId.
69          pickupLocation := aPickup.
70          dropoffLocation := aDropoff.
71          distance := aDistance.
72          ^self
73      ]
74
75      id [ ^rideID ]
76      distance [ ^distance ]
77
78      "Each subclass prices itself."
79      fare [ ^self subclassResponsibility ]
80
81      "The label a subclass reports in ride details."
82      rideType [ ^self subclassResponsibility ]
83
84      "The format is shared. The parts that vary are reached through the two
85       messages above, which are resolved against the receiver at run time."
86      rideDetails [
87          ^'[', rideID, '] ',
88            (RideFormat pad: self rideType to: 8), '  ',
89            pickupLocation, ' -> ', dropoffLocation, '  ',
90            (RideFormat miles: distance), '  ',
91            (RideFormat money: self fare)
92      ]
93  ]
94
```

Note. fare and rideType are declared subclassResponsibility, so each subclass must supply them. rideDetails is defined once and calls both, which is where the polymorphic behavior enters.

Figure 5

C++ Driver Class



Note. assignedRides is declared before the public label, so it is private. Every read and write goes through addRide, totalEarnings and getDriverInfo.